



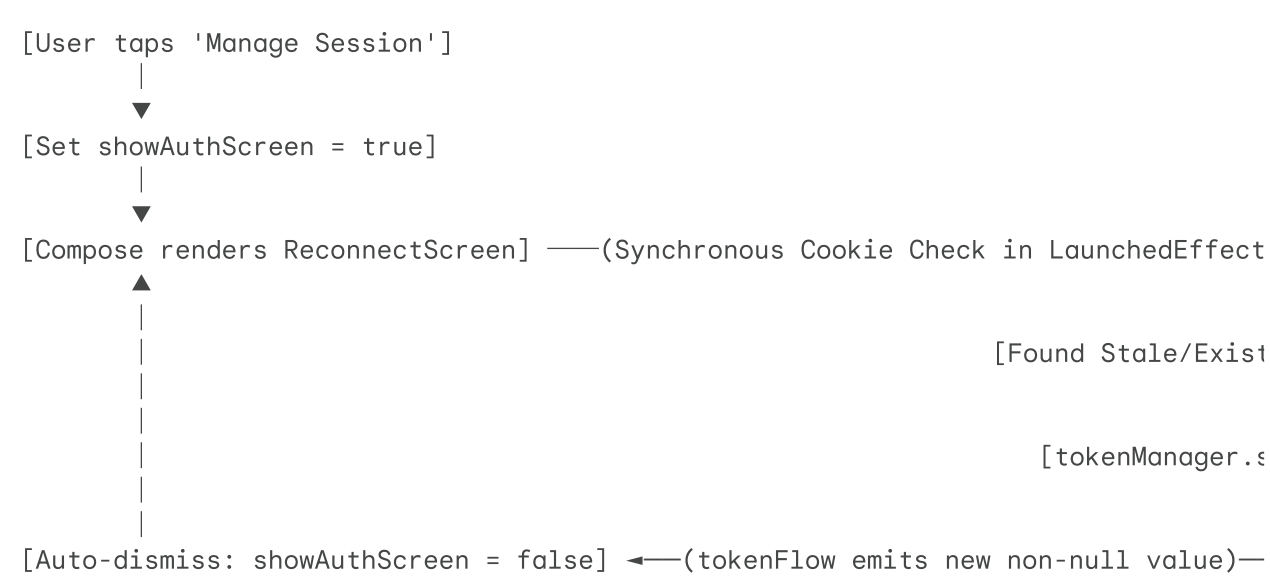
Resilient Session Management Architecture & Implementation

An enterprise-grade, fault-tolerant session lifecycle and authentication engine designed to eliminate race conditions, UI flashing, and native Chromium crashes (`onRenderProcessGone`) in Android applications interacting with web sessions.

1. Architectural Diagnostics & Problem Formulation

1.1 The State-Machine Auto-Dismiss Loop

In reactive UI architectures (Jetpack Compose), binding modal visibility directly to the presence of a non-null token creates a critical race condition when transitioning from an authenticated state to a management state:



Failure Mode: The entire roundtrip completes in 16–48 ms (1 to 3 frames). The user observes a black screen flash followed by immediate return to the settings panel.

1.2 WebView Process Partitioning & OS Fatalities

On Android 8.0 (API 26) and higher, WebView runs its rendering engine inside an isolated, sandboxed Zygote process (`chromium-renderer`).

- **Uncaught Termination:** Heavy Single Page Applications (Next.js, Svelte, Cloudflare Turnstile challenges) execute intensive JavaScript bundles that can exceed process memory ceilings. When the renderer crashes, the Android platform invokes `WebViewClient.onRenderProcessGone()` .
- **The Default Trap:** The default implementation of `onRenderProcessGone()` returns `false` . In Android, returning `false` instructs the OS to kill the host process (`android.os.Process.killProcess(Process.myPid())`). The entire app restarts abruptly.

- **Component Absence:** On custom de-Google ROMs (GrapheneOS, CalyxOS, LineageOS) or during Google Play background updates of "Android System WebView", calling `WebView(context)` throws `android.webkit.WebViewFactory$MissingWebViewPackageException` or `AndroidRuntimeException`. Uncaught, this causes an immediate application fatal crash.
- **Google OAuth & User-Agent Traps:** Google's OAuth 2.0 endpoint aggressively detects embedded WebViews via the `X-Requested-With` header and the `ww` token in the User-Agent string, returning HTTP 403 `disallowed_useragent`.

2. Four-Tier Fault Isolation Matrix

To guarantee zero crashes and complete user agency, the architecture divides session handling into four isolated recovery paths:

Tier	Component	Failure Recovery Vector	Primary Responsibility
Tier 1	Controlled Web Session Inspector	Stateful UI Guard (<code>SessionIntent</code>)	Inspect active JWT, decode claims, prevent automatic cookie re-harvesting.
Tier 2	Hardened Sandboxed WebView	<code>onRenderProcessGone</code> interceptor + <code>MissingWebViewPackageException</code> guard	In-app login with User-Agent spoofing and multi-window popup orchestration.
Tier 3	Chrome Custom Tabs (CCT) Bridge	External Sandboxed Browser + App Link Callback	Bypass Cloudflare bot-detection and Google OAuth restrictions using real device browser cookies.
Tier 4	Diagnostic Manual JWT Token Vault	Offline Base64URL Parser + Ping Validation	Manual token entry with claim validation (expiry, subject, issuer) and real-time backend verification.

3. Implementation Codebase

3.1 Session Intent & Sealed State Machine

```
data/local/preferences/JwtParser.kt
```

Zero-dependency, URL-safe Base64 parser for validating token structure and extracting expiration timestamps without external cryptographic libraries.

```
package com.zai.chat.data.local.preferences
```

```

import android.util.Base64
import kotlinx.serialization.json.Json
import kotlinx.serialization.json.JsonObject
import kotlinx.serialization.json.contentOrNull
import kotlinx.serialization.json.jsonPrimitive
import kotlinx.serialization.json.longOrNull
import java.nio.charset.StandardCharsets
import java.text.SimpleDateFormat
import java.util.Date
import java.util.Locale

data class JwtClaims(
    val rawToken: String,
    val subject: String?,
    val role: String?,
    val issuer: String?,
    val expiresAtEpochSeconds: Long?,
    val isExpired: Boolean,
    val humanReadableExpiry: String
)

object JwtParser {
    private val json = Json { ignoreUnknownKeys = true; isLenient = true }

    fun parse(rawToken: String?): JwtClaims? {
        if (rawToken.isNullOrEmpty()) return null
        val cleanToken = rawToken.removePrefix("Bearer ").trim()
        val parts = cleanToken.split(".")
        if (parts.size < 2) return null

        return try {
            val payloadBytes = Base64.decode(parts[1], Base64.URL_SAFE or Base64
            val payloadString = String(payloadBytes, StandardCharsets.UTF_8)
            val jsonObject = json.decodeFromString<JsonObject>(payloadString)

            val exp = jsonObject["exp"]?.jsonPrimitive?.longOrNull
            val sub = jsonObject["sub"]?.jsonPrimitive?.contentOrNull
                ?: jsonObject["id"]?.jsonPrimitive?.contentOrNull
                ?: jsonObject["email"]?.jsonPrimitive?.contentOrNull
            val role = jsonObject["role"]?.jsonPrimitive?.contentOrNull
            val iss = jsonObject["iss"]?.jsonPrimitive?.contentOrNull

            val nowSeconds = System.currentTimeMillis() / 1000
            val isExpired = exp != null && exp < nowSeconds

            val humanExpiry = if (exp != null) {
                val date = Date(exp * 1000)
                val sdf = SimpleDateFormat("MMM dd, yyyy HH:mm:ss", Locale.getDe
                if (isExpired) "Expired on ${sdf.format(date)}" else "Valid unti
            } else {
                "No expiration defined (Perpetual)"
            }

            JwtClaims(
                rawToken = cleanToken,
                subject = sub,

```

```

        role = role,
        issuer = iss,
        expiresAtEpochSeconds = exp,
        isExpired = isExpired,
        humanReadableExpiry = humanExpiry
    )
} catch (_: Exception) {
    null
}
}
}

```

ui/screens/session/SessionUiState.kt

State definitions driving the decoupled session management flow.

```

package com.zai.chat.ui.screens.session

import com.zai.chat.data.local.preferences.JwtClaims

sealed interface SessionMode {
    /** Passive reconnect triggered by an unexpected 401 HTTP response */
    data object BackgroundExpired : SessionMode

    /** Explicit user-driven session configuration from Settings */
    data object ActiveManagement : SessionMode
}

sealed interface SessionVerificationState {
    data object Idle : SessionVerificationState
    data object Testing : SessionVerificationState
    data class Valid(val message: String) : SessionVerificationState
    data class Invalid(val error: String) : SessionVerificationState
}

data class SessionUiState(
    val mode: SessionMode = SessionMode.ActiveManagement,
    val currentClaims: JwtClaims? = null,
    val isWebViewActive: Boolean = false,
    val isChromeCustomTabTriggered: Boolean = false,
    val manualTokenInput: String = "",
    val verificationState: SessionVerificationState = SessionVerificationState.Idle,
    val webViewCrashOccurred: Boolean = false,
    val webViewMissingError: Boolean = false,
    val userFacingNotice: String? = null
)

sealed interface SessionUiEvent {
    data class SetMode(val mode: SessionMode) : SessionUiEvent
    data object LaunchWebView : SessionUiEvent
    data object DismissWebView : SessionUiEvent
    data object ClearWebCookies : SessionUiEvent
    data object LaunchChromeCustomTab : SessionUiEvent
}

```

```

data class UpdateManualToken(val token: String) : SessionUiEvent
data object ApplyManualToken : SessionUiEvent
data object VerifyCurrentSession : SessionUiEvent
data object PurgeSession : SessionUiEvent
data object DismissNotice : SessionUiEvent
data object AcknowledgeRendererCrash : SessionUiEvent
}

```

ui/screens/session/SessionViewModel.kt

Manages token extraction, manual validation tests, and prevents race conditions.

```

package com.zai.chat.ui.screens.session

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.zai.chat.data.local.preferences.JwtParser
import com.zai.chat.data.local.preferences.TokenManager
import com.zai.chat.network.ZaiApiService
import dagger.hilt.android.lifecycle.HiltViewModel
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.update
import kotlinx.coroutines.launch
import kotlinx.coroutines.withContext
import javax.inject.Inject

@HiltViewModel
class SessionViewModel @Inject constructor(
    private val tokenManager: TokenManager,
    private val apiService: ZaiApiService
) : ViewModel() {

    private val _uiState = MutableStateFlow(SessionUiState())
    val uiState: StateFlow<SessionUiState> = _uiState.asStateFlow()

    init {
        loadActiveToken()
    }

    fun loadActiveToken() {
        val stored = tokenManager.getStoredToken()
        val claims = JwtParser.parse(stored)
        _uiState.update {
            it.copy(
                currentClaims = claims,
                manualTokenInput = stored.orEmpty(),
                verificationState = SessionVerificationState.Idle
            )
        }
    }
}

```

```

fun onEvent(event: SessionUiEvent) {
    when (event) {
        is SessionUiEvent.SetMode -> _uiState.update { it.copy(mode = event.)
        is SessionUiEvent.LaunchWebView -> _uiState.update {
            it.copy(isWebViewActive = true, webViewCrashOccurred = false)
        }
        is SessionUiEvent.DismissWebView -> _uiState.update { it.copy(isWebV.
        is SessionUiEvent.ClearWebCookies -> clearCookiesInternal()
        is SessionUiEvent.LaunchChromeCustomTab -> _uiState.update { it.copy
        is SessionUiEvent.UpdateManualToken -> _uiState.update { it.copy(man
        is SessionUiEvent.ApplyManualToken -> applyInputToken()
        is SessionUiEvent.VerifyCurrentSession -> verifyTokenConnectivity()
        is SessionUiEvent.PurgeSession -> purgeSessionInternal()
        is SessionUiEvent.DismissNotice -> _uiState.update { it.copy(userFac
        is SessionUiEvent.AcknowledgeRendererCrash -> _uiState.update {
            it.copy(webViewCrashOccurred = false, isWebViewActive = false)
        }
    }
}

fun handleTokenDiscoveredFromWeb(rawToken: String) {
    val claims = JwtParser.parse(rawToken)
    if (claims == null) {
        _uiState.update { it.copy(userFacingNotice = "Captured payload was n
        return
    }

    // Save new verified token
    tokenManager.saveToken(claims.rawToken)
    _uiState.update {
        it.copy(
            currentClaims = claims,
            isWebViewActive = false,
            manualTokenInput = claims.rawToken,
            userFacingNotice = "Session renewed successfully: ${claims.subje
        )
    }
}

fun handleRendererCrash() {
    _uiState.update {
        it.copy(
            isWebViewActive = false,
            webViewCrashOccurred = true,
            userFacingNotice = "Chromium rendering process exhausted memory (
        )
    }
}

fun handleMissingWebView() {
    _uiState.update {
        it.copy(
            isWebViewActive = false,
            webViewMissingError = true,
            userFacingNotice = "System WebView package is missing or disable

```

```

    )
}
}

private fun applyInputToken() {
    val clean = _uiState.value.manualTokenInput.removePrefix("Bearer ").trim
    val claims = JwtParser.parse(clean)
    if (claims == null) {
        _uiState.update { it.copy(verificationState = SessionVerificationSta
        return
    }

    tokenManager.saveToken(clean)
    _uiState.update {
        it.copy(
            currentClaims = claims,
            verificationState = SessionVerificationState.Valid("JWT applied.
        )
    }
    verifyTokenConnectivity()
}

private fun verifyTokenConnectivity() {
    viewModelScope.launch {
        _uiState.update { it.copy(verificationState = SessionVerificationSta
        try {
            val models = withContext(Dispatchers.IO) { apiService.getAvailab
            if (models.isNotEmpty()) {
                _uiState.update {
                    it.copy(verificationState = SessionVerificationState.Val
                }
            } else {
                _uiState.update {
                    it.copy(verificationState = SessionVerificationState.Inv
                }
            }
        } catch (e: Exception) {
            _uiState.update {
                it.copy(verificationState = SessionVerificationState.Invalid
            }
        }
    }
}

private fun clearCookiesInternal() {
    try {
        android.webkit.CookieManager.getInstance().removeAllCookies {
            android.webkit.CookieManager.getInstance().flush()
            _uiState.update { it.copy(userFacingNotice = "Browser cookies co
        }
    } catch (_: Exception) {}
}

private fun purgeSessionInternal() {
    tokenManager.clearToken()
    clearCookiesInternal()
}

```

```

        _uiState.update {
            it.copy(
                currentClaims = null,
                manualTokenInput = "",
                verificationState = SessionVerificationState.Idle,
                userFacingNotice = "Active session discarded."
            )
        }
    }
}

```

3.2 Sandboxed WebView & Chromium Crash Isolation

`ui/screens/session/SafeWebViewContainer.kt`

Encapsulates `WebView` instantiation in a fail-safe Compose wrapper that handles `onRenderProcessGone`, blocks 403 disallowed_useragent, captures tokens from both `localStorage` and `Cookie`, and supports popup windows.

```

package com.zai.chat.ui.screens.session

import android.annotation.SuppressLint
import android.content.Context
import android.graphics.Bitmap
import android.os.Message
import android.view.ViewGroup
import android.webkit.CookieManager
import android.webkit.RenderProcessGoneDetail
import android.webkit.WebChromeClient
import android.webkit.WebResourceRequest
import android.webkit.WebSettings
import android.webkit.WebView
import android.webkit.WebViewClient
import androidx.compose.foundation.background
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.statusBarsPadding
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.rounded.ArrowBack
import androidx.compose.material.icons.rounded.CleaningServices
import androidx.compose.material.icons.rounded.Refresh
import androidx.compose.material3.CircularProgressIndicator
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton

```



```

import androidx.compose.material3.LinearProgressIndicator
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.DisposableEffect
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableFloatStateOf
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.unit.dp
import androidx.compose.ui.viewinterop.AndroidView
import com.zai.chat.config.ZaiConfig
import com.zai.chat.ui.theme.TrueBlack

@SuppressLint("SetJavaScriptEnabled")
@Composable
fun SafeWebViewContainer(
    mode: SessionMode,
    onTokenExtracted: (String) -> Unit,
    onDismiss: () -> Unit,
    onRendererCrashed: () -> Unit,
    onMissingWebView: () -> Unit,
    modifier: Modifier = Modifier
) {
    var webViewInstance by remember { mutableStateOf<WebView?>(null) }
    var loadingProgress by remember { mutableFloatStateOf(0f) }
    var isPageLoading by remember { mutableStateOf(true) }

    DisposableEffect(Unit) {
        onDispose {
            webViewInstance?.let { ww ->
                try {
                    val parent = ww.parent as? ViewGroup
                    parent?.removeView(ww)
                    ww.stopLoading()
                    ww.loadUrl("about:blank")
                    ww.clearHistory()
                    ww.removeAllViews()
                    ww.destroy()
                } catch (_: Throwable) {}
            }
            webViewInstance = null
        }
    }
}

Surface(
    color = TrueBlack,
    modifier = modifier.fillMaxSize()
) {
    Column(modifier = Modifier.fillMaxSize().statusBarsPadding()) {
        // Header Action Bar
    }
}

```

```

Row(
    modifier = Modifier
        .fillMaxWidth()
        .padding(horizontal = 8.dp, vertical = 4.dp),
    verticalAlignment = Alignment.CenterVertically
) {
    IconButton(onClick = onDismiss) {
        Icon(imageVector = Icons.Rounded.ArrowBack, contentDescription = null)
    }

    Text(
        text = "Sign in to z.ai",
        style = MaterialTheme.typography.titleMedium,
        color = Color.White,
        modifier = Modifier.weight(1f)
    )

    IconButton(onClick = { webViewInstance?.reload() }) {
        Icon(imageVector = Icons.Rounded.Refresh, contentDescription = null)
    }

    IconButton(onClick = {
        CookieManager.getInstance().removeAllCookies(null)
        webViewInstance?.clearCache(true)
        webViewInstance?.loadUrl(ZaiConfig.BASE_URL)
    }) {
        Icon(imageVector = Icons.Rounded.CleaningServices, contentDescription = null)
    }
}

if (isPageLoading) {
    LinearProgressIndicator(
        progress = { loadingProgress },
        modifier = Modifier.fillMaxWidth().height(2.dp),
        color = MaterialTheme.colorScheme.primary,
        trackColor = Color.Transparent
    )
} else {
    Spacer(modifier = Modifier.height(2.dp))
}

Box(modifier = Modifier.fillMaxSize()) {
    AndroidView(
        modifier = Modifier.fillMaxSize(),
        factory = { context ->
            try {
                createSafeWebView(
                    context = context,
                    mode = mode,
                    onProgressUpdate = { progress ->
                        loadingProgress = progress / 100f
                        isPageLoading = progress < 100
                    },
                    onTokenFound = onTokenExtracted,
                    onRendererFatal = onRendererCrashed
                ).also { webViewInstance = it }
            }
        }
    )
}

```

```

        } catch (e: Exception) {
            onMissingWebView()
            WebView(context) // Degrade to empty view
        }
    }
}

}

}

}

}

}

private fun createSafeWebView(
    context: Context,
    mode: SessionMode,
    onProgressUpdate: (Int) -> Unit,
    onTokenFound: (String) -> Unit,
    onRendererFatal: () -> Unit
): WebView {
    val webView = WebView(context)

    with(webView.settings) {
        javaScriptEnabled = true
        domStorageEnabled = true
        databaseEnabled = true
        loadsImagesAutomatically = true
        mixedContentMode = WebSettings.MIXED_CONTENT_NEVER_ALLOW
        setSupportMultipleWindows(true)
        javaScriptCanOpenWindowsAutomatically = true

        // Clean Desktop User-Agent: Strip ';' and Android WebView identifier
        userAgentString = ZaiConfig.USER_AGENT
    }

    val cookieManager = CookieManager.getInstance()
    cookieManager.setAcceptCookie(true)
    cookieManager.setAcceptThirdPartyCookies(webView, true)

    webView.webChromeClient = object : WebChromeClient() {
        override fun onProgressChanged(view: WebView?, newProgress: Int) {
            onProgressUpdate(newProgress)
            if (newProgress > 60) {
                attemptTokenHarvest(view, mode, onTokenFound)
            }
        }
    }

    override fun onCreateWindow(
        view: WebView?,
        isDialog: Boolean,
        isUserGesture: Boolean,
        resultMsg: Message?
    ): Boolean {
        // Handle OAuth Popups inside the same frame without throwing crashes
        val transport = resultMsg?.obj as? WebView.WebViewTransport
        transport?.webView = view
        resultMsg?.sendToTarget()
        return true
    }
}

```

```

    }
}

webView.webViewClient = object : WebViewClient() {
    override fun onPageStarted(view: WebView?, url: String?, favicon: Bitmap?) {
        super.onPageStarted(view, url, favicon)
        attemptTokenHarvest(view, mode, onTokenFound)
    }

    override fun onPageFinished(view: WebView?, url: String?) {
        super.onPageFinished(view, url)
        attemptTokenHarvest(view, mode, onTokenFound)
    }

    override fun shouldOverrideUrlLoading(view: WebView?, request: WebResourceRequest?) {
        val destination = request?.url?.toString().orEmpty()

        // Trap OAuth callback loops or App links
        if (destination.contains("token=") || destination.contains("access_token=")) {
            val extracted = extractTokenFromUrl(destination)
            if (!extracted.isNullOrBlank()) {
                onTokenFound(extracted)
                return true
            }
        }
        return false
    }

    override fun onRenderProcessGone(view: WebView?, detail: RenderProcessGoneDetail?) {
        // CRITICAL OS RESILIENCE:
        // Returning true tells Android: DO NOT terminate the host app process
        try {
            val parent = view?.parent as? ViewGroup
            parent?.removeView(view)
            view?.destroy()
        } catch (_: Throwable) {}

        onRendererFatal()
        return true
    }
}

webView.loadUrl(ZaiConfig.BASE_URL)
return webView
}

private fun attemptTokenHarvest(
    view: WebView?,
    mode: SessionMode,
    onTokenFound: (String) -> Unit
) {
    if (view == null) return

    // 1. Check Web Storage (Open WebUI primary token container)
    view.evaluateJavascript(
        "(function() { try { return localStorage.getItem('token'); } catch(e) {"
    )
}

```

```

    ) { rawStorageToken ->
        val cleaned = rawStorageToken?.trim("'", ' ', '\\')
        if (!cleaned.isNullOrBlank() && cleaned != "null" && cleaned != "undefined") {
            onTokenFound(cleaned)
            return@evaluateJavascript
        }

        // 2. Fallback: Parse Cookie Jar for session tokens
        val cookies = CookieManager.getInstance().getCookie(ZaiConfig.BASE_URL)
        if (!cookies.isNullOrEmpty()) {
            val tokenMatch = cookies.split(";")
                .map { it.trim() }
                .firstOrNull { it.startsWith("token=") || it.startsWith("jwt=")
                    ?.substringAfter("=")

            if (!tokenMatch.isNullOrBlank() && tokenMatch != "null") {
                // In Active Management mode, ignore stale cookies on initial request
                if (mode != SessionMode.ActiveManagement) {
                    onTokenFound(tokenMatch)
                }
            }
        }
    }
}

private fun extractTokenFromUrl(url: String): String? {
    return try {
        val uri = android.net.Uri.parse(url)
        uri.getQueryParameter("token")
        ?: uri.getQueryParameter("access_token")
        ?: uri.fragment?.split("&")
            ?.map { it.trim() }
            ?.firstOrNull { it.startsWith("access_token=") }
            ?.substringAfter("access_token=")
    } catch (_: Exception) {
        null
    }
}

```

3.3 Chrome Custom Tabs Fallback Integration

network/auth/CustomTabAuthHelper.kt

Encapsulates launching Chrome Custom Tabs to bypass Cloudflare Turnstile CAPTCHA failures or Google OAuth restrictions on restricted Android environments.

```

package com.zai.chat.network.auth

import android.content.Context
import android.content.Intent
import android.net.Uri
import androidx.browser.customtabs.CustomTabColorSchemeParams
import androidx.browser.customtabs.CustomTabsIntent

```

```

import androidx.core.content.ContextCompat
import com.zai.chat.R
import com.zai.chat.config.ZaiConfig

object CustomTabAuthHelper {

    fun launch(context: Context) {
        val builder = CustomTabsIntent.Builder()

        // Match the application's true black OLED design system
        val colorScheme = CustomTabColorSchemeParams.Builder()
            .setToolbarColor(0xFF000000.toInt())
            .setNavigationBarColor(0xFF000000.toInt())
            .build()

        builder.setDefaultColorSchemeParams(colorScheme)
        builder.setShowTitle(true)
        builder.setShareState(CustomTabsIntent.SHARE_STATE_OFF)

        val customTabsIntent = builder.build()
        customTabsIntent.intent.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)

        try {
            customTabsIntent.launchUrl(context, Uri.parse(ZaiConfig.BASE_URL))
        } catch (_: Exception) {
            // Direct browser fallback if no Custom Tabs provider is available
            val fallback = Intent(Intent.ACTION_VIEW, Uri.parse(ZaiConfig.BASE_UI
                addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
            })
            context.startActivity(fallback)
        }
    }
}

```

3.4 Diagnostic Session Management & Inspector Screen

`ui/screens/session/ManageSessionScreen.kt`

The central administrative user interface that replaces auto-dismiss modals with an explicit, observable control plane.

```

package com.zai.chat.ui.screens.session

import android.content.ClipData
import android.content.ClipboardManager
import android.content.Context
import android.widget.Toast
import androidx.compose.animation.AnimatedVisibility
import androidx.compose.foundation.background
import androidx.compose.foundation.border
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column

```

```
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.imePadding
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.text.selection.SelectionContainer
import androidx.compose.foundation.verticalScroll
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.rounded.ArrowBack
import androidx.compose.material.icons.rounded.CheckCircle
import androidx.compose.material.icons.rounded.ContentCopy
import androidx.compose.material.icons.rounded.DeleteForever
import androidx.compose.material.icons.rounded.ErrorOutline
import androidx.compose.material.icons.rounded.Info
import androidx.compose.material.icons.rounded.Language
import androidx.compose.material.icons.rounded.Login
import androidx.compose.material.icons.rounded.OpenInBrowser
import androidx.compose.material.icons.rounded.Security
import androidx.compose.material.icons.rounded.Warning
import androidx.compose.material3.Button
import androidx.compose.material3.ButtonDefaults
import androidx.compose.material3.CircularProgressIndicator
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedButton
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.OutlinedTextFieldDefaults
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.material3.TopAppBar
import androidx.compose.material3.TopAppBarDefaults
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.text.font.FontFamily
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.zai.chat.network.auth.CustomTabAuthHelper
import com.zai.chat.ui.theme.ClaudePeach
import com.zai.chat.ui.theme.KimiCyan
```

```

import com.zai.chat.ui.theme.ObsidianBase
import com.zai.chat.ui.theme.SurfaceContainerDark
import com.zai.chat.ui.theme.SurfaceContainerHighDark
import com.zai.chat.ui.theme.TrueBlack

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun ManageSessionScreen(
    viewModel: SessionViewModel,
    onNavigateBack: () -> Unit,
    modifier: Modifier = Modifier
) {
    val state by viewModel.uiState.collectAsState()
    val context = LocalContext.current

    // Embedded Protected WebView Viewport
    if (state.isWebViewActive) {
        SafeWebViewContainer(
            mode = state.mode,
            onTokenExtracted = { token ->
                viewModel.handleTokenDiscoveredFromWeb(token)
            },
            onDismiss = { viewModel.onEvent(SessionUiEvent.DismissWebView) },
            onRendererCrashed = { viewModel.handleRendererCrash() },
            onMissingWebView = { viewModel.handleMissingWebView() }
        )
        return
    }

    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("Session & Authorization", style = MaterialTheme.typography.title2) },
                navigationIcon = {
                    IconButton(onClick = onNavigateBack) {
                        Icon(imageVector = Icons.Rounded.ArrowBack, contentDescription = "Back")
                    }
                },
                colors = TopAppBarDefaults.topAppBarColors(containerColor = TrueBlack)
            )
        },
        containerColor = TrueBlack,
        modifier = modifier.fillMaxSize().imePadding()
    ) { paddingValues ->
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(paddingValues)
                .verticalScroll(rememberScrollState())
                .padding(horizontal = 16.dp, vertical = 8.dp)
        ) {
            // Notice Banner
            state.userFacingNotice?.let { notice ->
                Surface(
                    color = SurfaceContainerHighDark,
                    shape = RoundedCornerShape(12.dp),
                    elevation = 4.dp
                ) {
                    Text(notice, style = MaterialTheme.typography.body1, color = TrueBlack)
                }
            }
        }
    }
}

```



```

        modifier = Modifier.fillMaxWidth().padding(bottom = 16.dp)
    ) {
        Row(
            modifier = Modifier.padding(12.dp),
            verticalAlignment = Alignment.CenterVertically
        ) {
            Icon(Icons.Rounded.Info, contentDescription = null, tint
            Spacer(modifier = Modifier.width(10.dp))
            Text(text = notice, style = MaterialTheme.typography.body
            IconButton(onClick = { viewModel.onEvent(SessionUiEvent.I
                Icon(Icons.Rounded.ArrowBack, contentDescription = "I
            }
        }
    }
}

```

```

// Crash Notification Recovery
if (state.webViewCrashOccurred) {
    Surface(
        color = ClaudePeach.copy(alpha = 0.15f),
        shape = RoundedCornerShape(12.dp),
        border = androidx.compose.foundation.BorderStroke(1.dp, Clau
        modifier = Modifier.fillMaxWidth().padding(bottom = 16.dp)
    ) {
        Column(modifier = Modifier.padding(14.dp)) {
            Row(verticalAlignment = Alignment.CenterVertically) {
                Icon(Icons.Rounded.Warning, contentDescription = nul
                Spacer(modifier = Modifier.width(8.dp))
                Text("Chromium Renderer Crash Caught", fontWeight = B
            }
            Spacer(modifier = Modifier.height(6.dp))
            Text(
                text = "The isolated web process terminated. Host ap
                style = MaterialTheme.typography.bodyMedium,
                color = Color.LightGray
            )
            Spacer(modifier = Modifier.height(8.dp))
            Button(
                onClick = { viewModel.onEvent(SessionUiEvent.Acknowl
                colors = ButtonDefaults.buttonColors(containerColor :
            ) {
                Text("Acknowledge", color = Color.Black)
            }
        }
    }
}

```

```

// SECTION 1: Active Token Inspector
Text("ACTIVE JWT STATUS", style = MaterialTheme.typography.labelSmall)
Spacer(modifier = Modifier.height(8.dp))
ActiveClaimsCard(
    claims = state.currentClaims,
    onCopyToken = {
        state.currentClaims?.rawToken?.let { token ->
            val cm = context.getSystemService(Context.CLIPBOARD_SERVICE)
            cm.setPrimaryClip(ClipData.newPlainText("Zai Token", token)
        }
    }
)

```

```

        Toast.makeText(context, "Token copied to clipboard", Toast.LENGTH_SHORT)
    }
}
)

Spacer(modifier = Modifier.height(24.dp))

// SECTION 2: Connection Testing
Text("CONNECTIVITY VERIFICATION", style = MaterialTheme.typography.h1)
Spacer(modifier = Modifier.height(8.dp))
Row(
    modifier = Modifier.fillMaxWidth(),
    horizontalArrangement = Arrangement.spacedBy(10.dp),
    verticalAlignment = Alignment.CenterVertically
) {
    Button(
        onClick = { viewModel.onEvent(SessionUiEvent.VerifyCurrentSession) },
        enabled = state.currentClaims != null && state.verifications.isNotEmpty(),
        colors = ButtonDefaults.buttonColors(containerColor = SurfaceColor),
        modifier = Modifier.weight(1f)
    ) {
        if (state.verifications.isNotEmpty()) {
            CircularProgressIndicator(modifier = Modifier.size(16.dp))
            Spacer(modifier = Modifier.width(8.dp))
            Text("Verifying...")
        } else {
            Icon(Icons.Rounded.Security, contentDescription = null, modifier = Modifier.size(16.dp))
            Spacer(modifier = Modifier.width(8.dp))
            Text("Test API Ping")
        }
    }

    OutlinedButton(
        onClick = { viewModel.onEvent(SessionUiEvent.PurgeSession) },
        colors = ButtonDefaults.outlinedButtonColors(contentColor = SurfaceColor),
        modifier = Modifier.weight(1f)
    ) {
        Icon(Icons.Rounded.DeleteForever, contentDescription = null, modifier = Modifier.size(16.dp))
        Spacer(modifier = Modifier.width(8.dp))
        Text("Clear Token")
    }
}

when (val v = state.verifications.firstOrNull()) {
    is SessionVerificationState.Valid -> {
        Text(text = v.message, color = Color(0xFF4CAF50), style = MaterialTheme.typography.h2)
    }
    is SessionVerificationState.Invalid -> {
        Text(text = v.error, color = ClaudePeach, style = MaterialTheme.typography.h2)
    }
    else -> {}
}

Spacer(modifier = Modifier.height(24.dp))

// SECTION 3: Sign In Actions (Web / Chrome Custom Tab)

```

```
Text("RE-AUTHENTICATION PROTOCOLS", style = MaterialTheme.typography
Spacer(modifier = Modifier.height(8.dp))
```

```
Surface(
    color = SurfaceContainerDark,
    shape = RoundedCornerShape(12.dp),
    modifier = Modifier.fillMaxWidth()
) {
    Column(modifier = Modifier.padding(14.dp)) {
        Text("Option A: Embedded Sandboxed Web", fontWeight = FontWe
        Text("Spawns an in-app WebView. Safe from OS termination. In
        Spacer(modifier = Modifier.height(10.dp))
        Button(
            onClick = {
                viewModel.onEvent(SessionUiEvent.SetMode(SessionMode
                viewModel.onEvent(SessionUiEvent.LaunchWebView)
            },
            colors = ButtonDefaults.buttonColors(containerColor = Cl
            modifier = Modifier.fillMaxWidth()
        ) {
            Icon(Icons.Rounded.Login, contentDescription = null, tin
            Spacer(modifier = Modifier.width(8.dp))
            Text("Open Sandboxed WebView", color = Color.Black, fontl
        }

        Spacer(modifier = Modifier.height(16.dp))

        Text("Option B: External Browser (Chrome Custom Tabs)", fontl
        Text("Recommended if Google OAuth 403 or Cloudflare Turnstil
        Spacer(modifier = Modifier.height(10.dp))
        OutlinedButton(
            onClick = { CustomTabAuthHelper.launch(context) },
            modifier = Modifier.fillMaxWidth()
        ) {
            Icon(Icons.Rounded.OpenInBrowser, contentDescription = n
            Spacer(modifier = Modifier.width(8.dp))
            Text("Launch Chrome Custom Tab")
        }
    }
}
```

```
Spacer(modifier = Modifier.height(24.dp))
```

```
// SECTION 4: Direct Manual Token Injection
```

```
Text("MANUAL JWT INJECTION", style = MaterialTheme.typography.labelSi
Spacer(modifier = Modifier.height(8.dp))
```

```
OutlinedTextField(
    value = state.manualTokenInput,
    onChange = { viewModel.onEvent(SessionUiEvent.UpdateManualT
    label = { Text("Paste Bearer Token or JWT") },
    placeholder = { Text("eyJhbGciOiJIUzI1NiIsIn...") },
    modifier = Modifier.fillMaxWidth(),
    textStyle = MaterialTheme.typography.bodyMedium.copy(fontFamily :
    shape = RoundedCornerShape(10.dp),
    colors = OutlinedTextFieldDefaults.colors(
```

```

        focusedBorderColor = ClaudePeach,
        unfocusedBorderColor = SurfaceContainerHighDark,
        focusedContainerColor = ObsidianBase,
        unfocusedContainerColor = ObsidianBase
    ),
    maxLines = 4
)

Spacer(modifier = Modifier.height(10.dp))

Button(
    onClick = { viewModel.onEvent(SessionUiEvent.ApplyManualToken) }
    enabled = state.manualTokenInput.isNotBlank(),
    colors = ButtonDefaults.buttonColors(containerColor = SurfaceCon
    modifier = Modifier.align(Alignment.End)
) {
    Text("Validate & Save Token")
}

Spacer(modifier = Modifier.height(32.dp))
}
}
}

@Composable
fun ActiveClaimsCard(
    claims: com.zai.chat.data.local.preferences.JwtClaims?,
    onCopyToken: () -> Unit
) {
    Surface(
        color = SurfaceContainerDark,
        shape = RoundedCornerShape(12.dp),
        modifier = Modifier.fillMaxWidth()
    ) {
        if (claims == null) {
            Column(modifier = Modifier.padding(16.dp)) {
                Row(verticalAlignment = Alignment.CenterVertically) {
                    Icon(Icons.Rounded.ErrorOutline, contentDescription = null,
                    Spacer(modifier = Modifier.width(8.dp))
                    Text("No Active Session Stored", fontWeight = FontWeight.Med
                }
            }
        } else {
            Column(modifier = Modifier.padding(16.dp)) {
                Row(
                    modifier = Modifier.fillMaxWidth(),
                    horizontalArrangement = Arrangement.SpaceBetween,
                    verticalAlignment = Alignment.CenterVertically
                ) {
                    Row(verticalAlignment = Alignment.CenterVertically) {
                        Box(
                            modifier = Modifier
                                .size(8.dp)
                                .clip(CircleShape)
                                .background(if (claims.isExpired) Color.Red else
                        )
                    }
                }
            }
        }
    }
}

```



```

import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.runtime.rememberCoroutineScope
import androidx.hilt.navigation.compose.hiltViewModel
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.compose.rememberNavController
import com.zai.chat.data.local.preferences.SettingsDataStore
import com.zai.chat.data.local.preferences.TokenManager
import com.zai.chat.network.auth.AuthEvent
import com.zai.chat.network.auth.AuthEventManager
import com.zai.chat.ui.screens.chat.ChatScreen
import com.zai.chat.ui.screens.chat.ChatViewModel
import com.zai.chat.ui.screens.drawer.DrawerChatList
import com.zai.chat.ui.screens.drawer.DrawerViewModel
import com.zai.chat.ui.screens.session.ManageSessionScreen
import com.zai.chat.ui.screens.session.SessionMode
import com.zai.chat.ui.screens.session.SessionUiEvent
import com.zai.chat.ui.screens.session.SessionViewModel
import com.zai.chat.ui.screens.settings.SettingsScreen
import com.zai.chat.ui.theme.ZaiTheme
import dagger.hilt.android.AndroidEntryPoint
import kotlinx.coroutines.launch
import javax.inject.Inject

@AndroidEntryPoint
class MainActivity : ComponentActivity() {

    @Inject lateinit var tokenManager: TokenManager
    @Inject lateinit var authEventManager: AuthEventManager
    @Inject lateinit var settingsDataStore: SettingsDataStore

    override fun onCreate(savedInstanceState: Bundle?) {
        enableEdgeToEdge()
        super.onCreate(savedInstanceState)

        setContent {
            val themeMode by settingsDataStore.themeMode.collectAsState(initial = SessionMode)
            val navController = rememberNavController()
            val drawerState = rememberDrawerState(initialValue = DrawerValue.Closed)
            val scope = rememberCoroutineScope()
            val sessionViewModel: SessionViewModel = hiltViewModel()

            // Handle background 401 token expirations gracefully via navigation
            LaunchedEffect(Unit) {
                authEventManager.events.collect { event ->
                    when (event) {
                        is AuthEvent.TokenExpired -> {
                            sessionViewModel.onEvent(SessionUiEvent.SetMode(SessionMode)) {
                                navController.navigate("session_management") {
                                    launchSingleTop = true
                                }
                            }
                        }
                    }
                }
            }
        }
    }
}

```

```

ZaiTheme(themeMode = themeMode) {
    val drawerViewModel: DrawerViewModel = hiltViewModel()

    ModalNavigationDrawer(
        drawerState = drawerState,
        gesturesEnabled = drawerState.isOpen,
        drawerContent = {
            DrawerChatList(
                viewModel = drawerViewModel,
                onSelectChat = { chatId ->
                    scope.launch { drawerState.close() }
                    navController.navigate("chat?chatId=$chatId") {
                        launchSingleTop = true
                    }
                },
                onNewChat = {
                    scope.launch { drawerState.close() }
                    navController.navigate("chat") {
                        launchSingleTop = true
                    }
                },
                onOpenSettings = {
                    scope.launch { drawerState.close() }
                    navController.navigate("settings")
                }
            )
        }
    ) {
        NavHost(navController = navController, startDestination = "chat") {
            composable("chat?chatId={chatId}") {
                val chatViewModel: ChatViewModel = hiltViewModel()
                ChatScreen(
                    viewModel = chatViewModel,
                    onOpenDrawer = { scope.launch { drawerState.open() } }
                )
            }
            composable("settings") {
                SettingsScreen(
                    onNavigateBack = { navController.popBackStack() }
                    onNavigateToSession = {
                        sessionViewModel.onEvent(SessionUiEvent.SetManagement)
                        navController.navigate("session_management")
                    }
                )
            }
            composable("session_management") {
                ManageSessionScreen(
                    viewModel = sessionViewModel,
                    onNavigateBack = { navController.popBackStack() }
                )
            }
        }
    }
}

```

```
}  
}
```

4. Verification Checklist & Regression Tests

Before deploying this architecture, verify each scenario manually or via UI tests:

1. Stale Cookie Regression Test:

- Ensure the app holds an active or expired JWT.
- Go to `Settings -> Manage Session`.
- **Expected:** Screen remains statically loaded on `Session & Authorization`. No flashing or sudden dismiss back to `Settings`.

2. Process Termination Recovery:

- Open Sandboxed WebView and trigger DevTools command `killProcess` or force memory exhaustion.